M19 # GitHub Actions + Claude Code

Methodology Advanced

Integrating Claude Code into GitHub CI/CD pipelines

Recommended pattern: externalized prompt

This pattern separates review logic from workflow mechanics. The YAML file orchestrates triggers and permissions; the `.github/prompts/code-review.md` file contains the review criteria. Modifying criteria does not require touching the workflow.

```
.github/
├── workflows/
│   └── claude-review.yml    # CI mechanics
└── prompts/
    └── code-review.md       # Criteria, protocols
```

This separation allows iterating on review quality without risking breaking the pipeline.

Authentication: OAuth vs API key

Method	Cost per review	Prerequisites
Subscription OAuth	Uses plan allowance; limits apply	Eligible account and OAuth token
`ANTHROPIC_API_KEY`	Token-based API billing	Anthropic API key

Choose authentication from the account and workload. A subscription review consumes allowance; neither method guarantees a fixed per-review cost.

Annotated minimal workflow

```yaml
on:
  pull_request:
    types: [opened, synchronize]
  issue_comment:
    types: [created]

jobs:
  claude-review:
    if: |
      github.event_name == 'pull_request' ||
      (github.event.issue.pull_request &&
       contains(github.event.comment.body, '/claude-review'))
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: anthropics/claude-code-action@v1
        with:
          claude_code_oauth_token:
${{ secrets.CLAUDE_CODE_OAUTH_TOKEN }}
          trigger_phrase: /claude-review
          prompt: Read .github/prompts/code-review.md and review the PR.
          claude_args: >-
            --model claude-sonnet-5-5
            --tools "Read,Glob,Grep"
```

`fetch-depth: 0` fetches full history. The current action accepts `prompt` and `claude_args`; the CLI tool restriction keeps this example read-only. See the official action inputs.

Allowed tools in CI

In CI, limit Claude to read-only tools. `Read`, `Glob` and `Grep` cover all needs of a code review. Adding `Write` or `Bash` creates a risk of accidentally modifying the repository during analysis. GitHub MCP tools (`mcp__github__get_pull_request_diff`, `mcp__github__submit_pending_pull_request_review`) allow Claude to post inline comments without direct write access to the repository.

Anti-hallucination protocol

Automated reviews suffer from a recurring problem: Claude invents line numbers or reports issues it has not verified. The mitigation protocol consists of explicitly asking in the prompt to verify before any assertion.

Wording in `code-review.md`:

```
Before reporting any issue, verify it with Read or Grep.
Never cite a line number you have not confirmed.
Structure output: MUST FIX / SHOULD FIX / CAN SKIP.
```

This instruction reduces false positives without complicating the workflow.

On-demand trigger: `/claude-review`

The `issue_comment` trigger allows any team member to trigger a review on demand by typing `/claude-review` in a PR comment. Useful for complex PRs where an automatic review on open would not have had enough context.

The condition `github.event.issue.pull_request != null` filters ordinary issue comments to activate the agent only on PRs.

Handling failures

Provide a fallback job that posts a comment if the Claude job fails. Without a fallback, a PR can remain silent for hours if the agent crashes, without the team knowing.

```yaml
- name: Handle review failure
  if: failure()
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.payload.pull_request.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '⚠️ Claude review failed — human reviewer needed.'
      });
```